Boilerplate Code Reduction with Lombok

```xml
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>
```

Hands-on Example

Let's suppose we need to define an event in a class. Specifically, our event will be characterized by:

- The type of event (e.g., create, delete, or update)
- A key identifying the data (e.g., a message ID)
- A data element (the actual data in the event)
- A timestamp indicating when the event occurred

To fully implement constructors, getters, setters, and other methods, we'd end up with something like this:

```java
public class Event<K, T> {
    public enum Type {CREATE, DELETE, UPDATE}
    private Type eventType;
    private K key;
    private T data;
    private ZonedDateTime eventCreatedAt;

    public Event() {}

    public Event(Type eventType, K key, T data) {
        this.eventType = eventType;
        this.key = key;
        this.data = data;
        this.eventCreatedAt = ZonedDateTime.now();
    }

    public Type getEventType() {
        return eventType;
    }

    public K getKey() {
        return key;
    }

    public T getData() {
        return data;
    }

    public ZonedDateTime getEventCreatedAt() {
        return eventCreatedAt;
    }

    @Override
    public String toString() {
        return "Event{" + "eventType=" + eventType + ", key=" + key + ", data=" + data + ", eventCreatedAt="
+ eventCreatedAt + '}';
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Event<?, ?> event = (Event<?, ?>) o;
        return eventType == event.eventType &&
                Objects.equals(key, event.key) &&
                Objects.equals(data, event.data) &&
                Objects.equals(eventCreatedAt, event.eventCreatedAt);
    }

    @Override
    public int hashCode() {
        return Objects.hash(eventType, key, data, eventCreatedAt);
    }
}
```

Creating a random Event<String, Integer> object looks like:

```java
int index = RANDOM.nextInt(Event.Type.class.getEnumConstants().length);
Event<String, Integer> event = new Event(
    Event.Type.class.getEnumConstants()[index],
    UUID.randomUUID().toString(),
    RANDOM.nextInt(100)
);
```

This process is tedious and cluttered, affecting readability. IDEs can auto-generate code for you, but this still doesn't address the underlying issue of verbose code. Enter Lombok!

@NoArgsConstructor, @AllArgsConstructor, and @RequiredArgsConstructor

Lombok provides annotations to avoid manually writing constructors:

- `@NoArgsConstructor` generates a no-arguments constructor.
- `@AllArgsConstructor` generates a constructor with one parameter for each field.
- `@RequiredArgsConstructor` generates a constructor for all uninitialized final fields and fields marked `@NonNull`.

Here's how our class looks after using Lombok constructors:

```java
@NoArgsConstructor
@AllArgsConstructor
public class Event<K, T> {
    public enum Type {CREATE, DELETE, UPDATE}
    private Type eventType;
    private K key;
    private T data;
    private ZonedDateTime eventCreatedAt;

    public Type getEventType() {
        return eventType;
    }

    public K getKey() {
        return key;
    }

    public T getData() {
        return data;
    }

    public ZonedDateTime getEventCreatedAt() {
        return eventCreatedAt;
    }

    @Override
    public String toString() {
        return "Event{" + "eventType=" + eventType + ", key=" + key + ", data=" + data + ", eventCreatedAt="
+ eventCreatedAt + '}';
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Event<?, ?> event = (Event<?, ?>) o;
        return eventType == event.eventType &&
                Objects.equals(key, event.key) &&
                Objects.equals(data, event.data) &&
                Objects.equals(eventCreatedAt, event.eventCreatedAt);
    }

    @Override
    public int hashCode() {
        return Objects.hash(eventType, key, data, eventCreatedAt);
    }
}
```

@Getter and @Setter

The `@Getter` and `@Setter` annotations auto-generate getters and setters for all fields, further simplifying the code:

```java
@NoArgsConstructor
@AllArgsConstructor
@Getter
@Setter
public class Event<K, T> {
    public enum Type {CREATE, DELETE, UPDATE}
    private Type eventType;
    private K key;
    private T data;
    private ZonedDateTime eventCreatedAt;

    @Override
    public String toString() {
        return "Event{" + "eventType=" + eventType + ", key=" + key + ", data=" + data + ", eventCreatedAt="
+ eventCreatedAt + '}';
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Event<?, ?> event = (Event<?, ?>) o;
        return eventType == event.eventType &&
                Objects.equals(key, event.key) &&
                Objects.equals(data, event.data) &&
                Objects.equals(eventCreatedAt, event.eventCreatedAt);
    }

    @Override
    public int hashCode() {
        return Objects.hash(eventType, key, data, eventCreatedAt);
    }
}
```

@ToString and @EqualsAndHashCode

```java
@NoArgsConstructor
@AllArgsConstructor
@Getter
@Setter
@EqualsAndHashCode
@ToString
public class Event<K, T> {
    public enum Type {CREATE, DELETE, UPDATE}
    private Type eventType;
    private K key;
    private T data;
    private ZonedDateTime eventCreatedAt;
}
```

@Data

For even greater simplicity, the `@Data` annotation bundles `@ToString`, `@EqualsAndHashCode`, `@Getter`, `@Setter`, and `@RequiredArgsConstructor`. Our class can now be rewritten as:

```java
@NoArgsConstructor
@AllArgsConstructor
@Data
public class Event<K, T> {
    public enum Type {CREATE, DELETE, UPDATE}
    @NonNull private Type eventType;
    private K key;
    private T data;
    private ZonedDateTime eventCreatedAt;
}
```

Resources

- https://projectlombok.org/